C Programming - Detailed Notes

(Unit 2)

Operators and Expressions

1. Operators - Introduction

An operator is a symbol that tells the compiler to perform a specific mathematical, relational, or logical operation on one or more operands (variables/values).

Example:

```
int a = 10, b = 5;
int sum = a + b;   // '+' is the operator, a and b are operands
```

C operators are broadly classified as:

Category	Operators
Arithmetic	`+ - * / %`
Relational	`> < >= <= == !=`
Logical	`&& || !`
Assignment	`= += -= *= /= %= etc.`
Increment/Decrement	`++ --`
Conditional (Ternary)	`?:`
Bitwise	`& | ^ ~ << >>`
Special	`sizeof, &, *, , . ->`

2. Arithmetic Operators

Used to perform basic mathematical operations.

Operator	Meaning	Example (a=10, b=3)	Result
`+`	Addition	`a + b`	13
`-`	Subtraction	`a - b`	7
`*`	Multiplication	`a * b`	30
`/`	Division	`a / b`	3 (integer division)
`%`	Modulus (remainder)	`a % b`	1

Note: `%` (modulus) works only with integer operands, not with float/double. Integer division truncates the decimal part: `10 / 3` gives 3, not 3.33.

Example Program:

```
#include <stdio.h>
int main() {
    int a = 10, b = 3;
    printf("Addition:       %d\n", a + b);   // 13
    printf("Subtraction:    %d\n", a - b);   // 7
    printf("Multiplication: %d\n", a * b);   // 30
    printf("Division:       %d\n", a / b);   // 3
    printf("Modulus:        %d\n", a % b);   // 1
    return 0;
}
```

Unary Arithmetic Operators:

```
int a = 5;
int b = -a;   // unary minus, b = -5
int c = +a;   // unary plus,  c =  5 (rarely used)
```

3. Relational Operators

Used to compare two values. The result is always either 1 (true) or 0 (false) in C.

Operator	Meaning	Example (a=10, b=20)	Result
>	Greater than	a > b	0 (false)
<	Less than	a < b	1 (true)
>=	Greater than or equal to	a >= 10	1
<=	Less than or equal to	a <= 5	0
==	Equal to	a == b	0
!=	Not equal to	a != b	1

Example Program:

```
#include <stdio.h>
int main() {
    int a = 10, b = 20;
    printf("%d\n", a > b);    // 0
    printf("%d\n", a < b);    // 1
    printf("%d\n", a == b);   // 0
    printf("%d\n", a != b);   // 1
    return 0;
}
```

Common Mistake: Using = (assignment) instead of == (comparison) inside conditions.

```
if (a = 5)    // WRONG:   assigns 5 to a, always true
if (a == 5)   // CORRECT: compares a with 5
```

4. Logical Operators

Used to combine two or more relational expressions / conditions. Result is 1 (true) or 0 (false).

Operator	Name	Meaning
&&	Logical AND	True only if both conditions are true
||	Logical OR	True if at least one condition is true
!	Logical NOT	Reverses the result (true -> false, false -> true)

Truth Table:

A	B	A && B	A || B	!A
1	1	1	1	0
1	0	0	1	0
0	1	0	1	1
0	0	0	0	1

Example Program:

```
#include <stdio.h>
int main() {
    int age = 25;
    int hasID = 1;

    if (age >= 18 && hasID == 1)
        printf("Entry allowed\n");
    else
        printf("Entry denied\n");

    if (age < 18 || hasID == 0)
        printf("Some condition true\n");

    printf("!hasID = %d\n", !hasID);   // 0
    return 0;
}
```

Short-circuit evaluation: In `A && B`, if A is false, B is never evaluated. In `A || B`, if A is true, B is never evaluated. This is important when B has side effects (like a function call).

5. Assignment Operators

Used to assign a value to a variable. C provides compound assignment operators as a shorthand.

Operator	Example	Equivalent To
=	a = 5	Assigns 5 to a
+=	a += 5	a = a + 5
-=	a -= 5	a = a - 5
*=	a *= 5	a = a * 5
/=	a /= 5	a = a / 5
%=	a %= 5	a = a % 5

Example Program:

```
#include <stdio.h>
int main() {
    int a = 10;
    a += 5;   // a = 15
    printf("%d\n", a);
    a -= 3;   // a = 12
    printf("%d\n", a);
    a *= 2;   // a = 24
    printf("%d\n", a);
    a /= 4;   // a = 6
    printf("%d\n", a);
    a %= 4;   // a = 2
    printf("%d\n", a);
    return 0;
}
```

6. Increment and Decrement Operators

Used to increase or decrease the value of a variable by 1. These are unary operators.

Operator	Meaning
`++`	Increment by 1
`--`	Decrement by 1

Two forms:

Form	Syntax	Behavior
Pre-increment/decrement	`++a, --a`	Value is changed first, then used in expression
Post-increment/decrement	`a++, a--`	Value is used first in expression, then changed

Example Program (illustrating the difference):

```
#include <stdio.h>
int main() {
    int a = 5, b;

    b = ++a;   // pre-increment: a becomes 6, then b = 6
    printf("a=%d b=%d\n", a, b);   // a=6 b=6

    a = 5;
    b = a++;   // post-increment: b = 5 (old value), then a becomes 6
    printf("a=%d b=%d\n", a, b);   // a=6 b=5

    return 0;
}
```

Common use in loops:

```
for (int i = 0; i < 5; i++) {
    printf("%d ", i);
}
// Output: 0 1 2 3 4
```

7. Conditional Operator (Ternary Operator)

The only operator in C that takes three operands. It is a compact alternative to if-else.

Syntax:

```
condition ? expression_if_true : expression_if_false;
```

Example Program:

```
#include <stdio.h>
int main() {
    int a = 10, b = 20, max;
    max = (a > b) ? a : b;
    printf("Maximum = %d\n", max);   // Maximum = 20

    // Odd/Even check using ternary operator
    int n = 7;
    printf("%s\n", (n % 2 == 0) ? "Even" : "Odd");   // Odd
    return 0;
}
```

Equivalent if-else:

```
if (a > b)
    max = a;
else
    max = b;
```

Nested ternary (used sparingly, readability suffers):

```
int a = 5, b = 10, c = 15;
int largest = (a > b) ? ((a > c) ? a : c) : ((b > c) ? b : c);
printf("%d\n", largest);   // 15
```

8. Bitwise Operators

Operate directly on the individual bits of integer data types (int, char).

Operator	Name	Description
&	Bitwise AND	1 only if both bits are 1
|	Bitwise OR	1 if at least one bit is 1
^	Bitwise XOR	1 if bits are different
~	Bitwise NOT (complement)	Inverts all bits
<<	Left shift	Shifts bits left, fills with 0
>>	Right shift	Shifts bits right

Example: a = 5 (0101), b = 3 (0011)

Operation	Binary Result	Decimal Result
`a & b`	`0001`	1
`a | b`	`0111`	7
`a ^ b`	`0110`	6
`~a`	`...11111010`	-6 (2's complement)
`a << 1`	`1010`	10
`a >> 1`	`0010`	2

Bit Position

```
bit position:  3  2  1  0
place value:   8  4  2  1
```

Example Program:

```c
#include <stdio.h>
int main() {
    int a = 5, b = 3;
    // a = 0101 (5)
    // b = 0011 (3)

    printf("a & b  = %d\n", a & b);    // AND: 1 only if both bits are 1
                                       //   0101
                                       // & 0011
                                       //   ----
                                       //   0001 = 1

    printf("a | b  = %d\n", a | b);    // OR: 1 if either bit is 1
                                       //   0101
                                       // | 0011
                                       //   ----
                                       //   0111 = 7

    printf("a ^ b  = %d\n", a ^ b);    // XOR: 1 only if bits differ
                                       //   0101
                                       // ^ 0011
                                       //   ----
                                       //   0110 = 6

    printf("~a     = %d\n", ~a);       // NOT: flips every bit
                                       //   ~0101 = 1010 (in 4 bits)
                                       //   in 32-bit two's complement
                                       //   this equals -(5 + 1) = -6

    printf("a << 1 = %d\n", a << 1);   // Left shift: move bits left by 1
                                       //   0101 -> 1010 = 10
                                       //   (same as 5 * 2)

    printf("a >> 1 = %d\n", a >> 1);   // Right shift: move bits right by 1
                                       //   0101 -> 0010 = 2
                                       //   (same as 5 / 2, rounded down)

    return 0;
}
```

Practical use: Left shift by n multiplies by 2^n; right shift by n divides by 2^n (for positive integers). This is often used for fast multiplication/division and for setting/clearing/checking specific bits (flags, masks) in embedded and systems programming.

9. Special Operators

Operator	Name	Purpose
`sizeof()`	Size-of operator	Returns size (in bytes) of a data type/variable
`&`	Address-of operator	Returns the memory address of a variable
`*`	Pointer/indirection operator	Declares a pointer / accesses value at an address
`,`	Comma operator	Evaluates multiple expressions, returns the last one
`.`	Member (dot) operator	Accesses a member of a structure/union
`->`	Arrow operator	Accesses a structure member via a pointer

Example Program:

```
#include <stdio.h>
int main() {
    int a = 10;
    int *ptr = &a;   // '&' gets address of a; '*' declares a pointer

    printf("Size of int: %lu bytes\n", sizeof(int));   // 4
    printf("Address of a: %p\n", &a);
    printf("Value via pointer: %d\n", *ptr);           // 10

    int x, y;
    x = (y = 5, y + 10);   // comma operator: y = 5, then x = 15
    printf("x = %d, y = %d\n", x, y);
    return 0;
}
```

10. Arithmetic Expressions

An expression is a combination of variables, constants, and operators that evaluates to a single value.

Types of expressions in C:

Type	Example
Arithmetic expression	`a + b * c`
Relational expression	`a > b`
Logical expression	`a > b && c < d`
Mixed-mode expression	`a + 5.5 (int + float)`

Example:

```
int a = 5, b = 3, c = 2;
int result = a + b * c;   // = 5 + (3*2) = 11
```

Mixed-mode arithmetic: When an expression contains both int and float/double, C automatically converts (promotes) the int to float/double before evaluating. This is called *implicit type conversion*.

```
int a = 5;
float b = 2.0;
float result = a / b;   // a is promoted to float => 2.5
```

11. Evaluation of Expressions

Expressions are evaluated based on operator precedence and associativity (see Section 12). Evaluation converts an expression into a single value by applying operators in the correct order.

Example - step-by-step evaluation:

```
int result = 2 + 3 * 4 - 6 / 2;
```

Evaluation steps:

```
Step 1:  3 * 4 = 12     (multiplication first)
Step 2:  6 / 2 = 3      (division next)
Step 3:  2 + 12 = 14
Step 4:  14 - 3 = 11

Final result = 11
```

Program:

```
#include <stdio.h>
int main() {
    int result = 2 + 3 * 4 - 6 / 2;
    printf("Result = %d\n", result);   // Result = 11
    return 0;
}
```

Evaluation of expressions with parentheses (highest priority):

```
int result = (2 + 3) * (4 - 1);   // = 5 * 3 = 15
```

12. Precedence of Arithmetic Operators

When an expression has multiple arithmetic operators, precedence decides which operator is evaluated first.

Precedence	Operators	Associativity
1 (Highest)	( )	Left to right
2	*, /, %	Left to right
3 (Lowest)	+, -	Left to right

Example:

```
int result = 10 + 20 * 30;
// * has higher precedence than +
// = 10 + 600 = 610

int result = 100 / 10 * 2;
// / and * have SAME precedence -> evaluated left to right
// = (100 / 10) * 2 = 10 * 2 = 20
```

13. Operator Precedence and Associativity (Complete Table)

Precedence determines which operator is applied first when different operators appear together. Associativity determines the direction of evaluation (left-to-right or right-to-left) when operators of the same precedence appear together.

Precedence (High -> Low)	Operators	Associativity
1	`() [] -> .`	Left to Right
2	`! ~ ++ -- +(unary) -(unary) *(pointer) &(address) sizeof`	Right to Left
3	`* / %`	Left to Right
4	`+ -`	Left to Right
5	`<< >>`	Left to Right
6	`< <= > >=`	Left to Right
7	`== !=`	Left to Right
8	`& (bitwise AND)`	Left to Right
9	`^ (bitwise XOR)`	Left to Right
10	`| (bitwise OR)`	Left to Right
11	`&&`	Left to Right
12	`||`	Left to Right
13	`?: (ternary)`	Right to Left
14	`= += -= *= /= %= etc.`	Right to Left
15 (Lowest)	`, (comma)`	Left to Right

Example - Precedence in action:

```
int a = 5, b = 10, c = 15, result;
result = a + b * c;
// '*' (precedence 3) evaluated before '+' (precedence 4)
// = 5 + 150 = 155
```

Example - Associativity in action (same precedence):

```
int a = 20, b = 10, c = 5;
int result = a - b - c;
// '-' is left-to-right associative
// = (a - b) - c = (20 - 10) - 5 = 5
// NOT a - (b - c), which would give 15
```

Example - Assignment is right-to-left associative:

```
int a, b, c;
a = b = c = 10;
// evaluated as: a = (b = (c = 10))
// c = 10, then b = 10, then a = 10
```

Example combining multiple operator types:

```
#include <stdio.h>
int main() {
    int a = 5, b = 10, c = 15;
    int result = a + b > c && c > 0;
    // '+' binds tighter than '>', and '>' binds tighter than '&&'
    // Step 1: a + b  -> 5 + 10 = 15
    // Step 2: 15 > c -> 15 > 15 -> 0
    // Step 3: c > 0  -> 15 > 0  -> 1
    // Step 4: 0 && 1 -> 0
    printf("%d\n", result);   // 0
    return 0;
}
```

14. Mathematical (Library) Functions

C provides many ready-made mathematical functions in the `math.h` header file, which must be included to use them. Programs using these must often be compiled with the `-lm` flag on some compilers (e.g., GCC on Linux: `gcc file.c -lm`).

Function	Purpose	Example	Result
`sqrt(x)`	Square root	`sqrt(25)`	5.0
`pow(x, y)`	x raised to power y	`pow(2, 3)`	8.0
`abs(x)`	Absolute value (integer)	`abs(-5)`	5
`fabs(x)`	Absolute value (float/double)	`fabs(-5.5)`	5.5
`ceil(x)`	Round up to nearest integer	`ceil(4.2)`	5.0
`floor(x)`	Round down to nearest integer	`floor(4.8)`	4.0
`log(x)`	Natural logarithm (base e)	`log(2.718)`	~1.0
`log10(x)`	Logarithm base 10	`log10(100)`	2.0
`sin(x)`, `cos(x)`, `tan(x)`	Trigonometric functions (x in radians)	`sin(0)`	0.0
`exp(x)`	e raised to power x	`exp(1)`	~2.718

Example Program:

```
#include <stdio.h>
#include <math.h>

int main() {
    double num = 25.0;

    // sqrt: the number that multiplied by itself gives 25 -> 5 x 5 = 25
    printf("Square root of %.1f = %.2f\n", num, sqrt(num));       // 5.00

    // pow(base, exponent): 2 x 2 x 2 = 8
    printf("2 raised to power 3 = %.2f\n", pow(2, 3));            // 8.00

    // ceil: rounds UP to the next whole number (4.2 -> 5)
    printf("Ceil of 4.2 = %.2f\n", ceil(4.2));                    // 5.00

    // floor: rounds DOWN to the previous whole number (4.8 -> 4)
    printf("Floor of 4.8 = %.2f\n", floor(4.8));                  // 4.00

    // fabs: removes the minus sign (absolute value) -> -7.5 becomes 7.5
    // use fabs for decimals; plain abs is for ints
    printf("Absolute value of -7.5 = %.2f\n", fabs(-7.5));        // 7.50

    // log10: "10 to what power gives 1000?" -> 10^3 = 1000, so 3
    printf("Log base 10 of 1000 = %.2f\n", log10(1000));          // 3.00

    return 0;
}
```

Compilation note (GCC/Linux):

```
gcc program.c -o program -lm
./program
```

Quick Revision Summary

Concept	One-line takeaway
Arithmetic Operators	+ - * / % : basic math; % only for integers
Relational Operators	> < >= <= == != : compare values, return 1/0
Logical Operators	&& || ! : combine conditions, support short-circuiting
Assignment Operators	= += -= *= /= %= : assign & compound update values
Increment/Decrement	++ -- : pre form changes first, post form changes after use
Conditional Operator	?: : compact replacement for simple if-else
Bitwise Operators	& | ^ ~ << >> : operate on individual bits
Special Operators	sizeof, &, *, ",", ., -> : memory, structures, pointers
Arithmetic Expressions	Combination of operators/operands evaluating to one value
Evaluation of Expressions	Follows precedence & associativity rules step-by-step
Precedence of Arithmetic Ops	() > * / % > + -
Precedence & Associativity	Full ranked table decides evaluation order for all operators
Mathematical Functions	sqrt, pow, abs, fabs, ceil, floor, log, sin/cos/tan from math.h